



Escuela de Ingeniería Informática

Escuela de Ingeniería Informática
School of Computer Science Engineering

Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Sistemas Distribuidos e Internet

Introducción a Node.js

Parte 1

Desarrollo de aplicaciones y servicios web modernos
con JavaScript



Dr. Edward Rolando Núñez Valdez

nunezedward@uniovi.es

Resultados de Aprendizaje

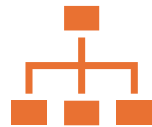
- Al finalizar este tema el estudiante será capaz de:
 - Explicar qué es Node.js y su modelo de ejecución basado en eventos.
 - Diferenciar comunicación síncrona y asíncrona.
 - Comprender el modelo Single Thread + Event Loop.
 - Crear una aplicación básica con Node.js.
 - Utilizar NPM para gestionar dependencias.
 - Desarrollar servicios web básicos usando Express.
 - Gestionar rutas, parámetros HTTP y vistas dinámicas.

Índice



Fundamentos de Node.js

¿Qué es Node.js?
Arquitectura basada en eventos
Modelo asíncrono y Event Loop
Ventajas y escalabilidad



Gestión de paquetes y entorno de desarrollo

NPM
package.json
Entorno de desarrollo



Desarrollo de aplicaciones

Servidor HTTP básico
Gestión de dependencias
Despliegue local



Desarrollo con Express

Instalación
Routing
Petición GET y POST
Middleware
Recursos estáticos
Motores de plantillas

¿Qué es Node.js?

- Node.js es un **entorno de ejecución (runtime)** que permite ejecutar **JavaScript fuera del navegador**, principalmente del lado del servidor.
- Está **construido sobre** el motor **V8** de Google Chrome.
- Node.js proporciona:
 - Modelo de ejecución basado en eventos
 - Operaciones I/O asíncronas y no bloqueantes
 - Módulos integrados (HTTP, FS, Path, etc.)



¿Por qué Node.js se considera también como una plataforma?

- Porque incluye un ecosistema que facilita el desarrollo:
 - Sistema de paquetes (NPM)
 - Gran comunidad
 - Librerías externas
 - Frameworks como Express, nestJS, Koa.js, etc

¿Por qué surge Node.js?

Problema en servidores tradicionales

- En muchos servidores clásicos:
 - Se crea un hilo por cada petición
 - Alto consumo de memoria
 - Dificultad para manejar miles de conexiones simultáneas
 - Escalabilidad
 - Costos

Propuesta de Node.js

- Un único hilo de ejecución
- Modelo basado en eventos (Event Loop)
- Operaciones I/O asíncronas y no bloqueantes
- Enfoque orientado a aplicaciones de red

Resultado: Mayor eficiencia para gestionar muchas conexiones concurrentes con menor consumo de recursos.

Origen y evolución de Node.js



- **Origen**

- Creado en 2009 por **Ryan Dahl**
- Diseñado para mejorar el rendimiento de aplicaciones orientadas a red
- Inicialmente respaldado por Joyent, empresa especializada en virtualización y computación en la nube.

- **Evolución**

- Crecimiento rápido gracias a su modelo asíncrono
- En 2015 pasa a ser gestionado por la **Node.js Foundation**
- Actualmente mantenido por la **OpenJS Foundation**
- En 2018, Ryan Dahl crea **Deno**, un nuevo runtime con:
 - Seguridad mejorada por defecto
 - Soporte nativo para TypeScript
 - Sistema de módulos moderno



Node.js Vs Deno

Característica	Node.js	Deno
Lenguaje principal	JavaScript y TypeScript (con configuración adicional)	Soporte nativo TypeScript y JavaScript
Motor de ejecución	Basado en el motor V8 de Google	Basado en V8, pero con Rust en su núcleo
Seguridad	Sin restricciones por defecto	Seguridad integrada: permisos explícitos para archivos, red y entorno
Sistema de módulos	CommonJS (requiere node_modules) y ES Modules	ES Modules
Gestión de dependencias	NPM (Node Package Manager)	Importación directa desde URL; no requiere un gestor externo
Ecosistema	Maduro y extenso	En desarrollo, menos paquetes
Peticiones Asíncronas	Callback/Promises	Async/Await (moderno)
Herramientas integradas	Necesita herramientas externas para formatear y testear	Incluye formateador, linter y herramientas de prueba integradas
Ejecución de código	Permite ejecutar cualquier cosa.	Requiere permisos explícitos para acceder a recursos
Comunidad	Grande y activa	Creciente, pero más pequeña
Rendimiento	Optimizado para aplicaciones grandes	Similar, pero con enfoque en seguridad
Licencia	MIT	MIT

¿Qué NO es Node.js?

- No es un lenguaje de programación
 - El lenguaje es **JavaScript**.
 - Node.js es el entorno que lo ejecuta.
- No es un Framework
 - No impone una arquitectura específica.
 - Frameworks como Express se ejecutan sobre Node.js
- No es un servidor web tradicional
 - No es como Apache o IIS.
 - Incluye módulos (como http) que permiten crear servidores web.
- No es un entorno multihilo por defecto
 - Utiliza un único hilo de ejecución con un modelo basado en eventos



Principales ventajas de Node.js

- **Unificación del lenguaje**
 - JavaScript en frontend y backend
 - Simplifica desarrollo Full Stack
 - Reduce curva de aprendizaje
- **Alto rendimiento**
 - Basado en el motor V8
 - Compilación **Just-In-Time (JIT)** que optimiza el código en tiempo de ejecución
- **Ideal para aplicaciones en tiempo real**
 - Chats
 - Streaming
 - APIs de alta concurrencia
 - Herramientas colaborativas



Ventajas en el desarrollo moderno

- **Ecosistema NPM**

- Amplio repositorio de paquetes reutilizables
- Acelera el desarrollo
- Permite instalar y gestionar dependencias fácilmente

- **Multiplataforma y Open Source**

- Compatible con Windows, Linux y macOS

- **Comunidad activa y mantenimiento continuo**



Ventajas arquitectónicas de Node.js

- **Escalabilidad horizontal**
 - Permite añadir múltiples instancias
 - Compatible con balanceadores de carga
- **Compatible con arquitecturas de microservicios**
 - Servicios ligeros
 - Despliegue independiente
- **Integración natural con entornos Cloud**
 - AWS, Azure, GCP



Escalabilidad en aplicaciones

- ¿Qué significa escalar?
 - Capacidad de una aplicación para manejar un aumento de carga o usuarios sin degradar su rendimiento.
- Existen dos formas principales:
 - Escalabilidad vertical
 - Escalabilidad horizontal

Escalabilidad vertical vs horizontal

Escalabilidad Horizontal	Escalabilidad Vertical
Agregar más instancias o servidores	Mejorar la máquina existente (CPU, RAM)
Ideal para entornos distribuidos y en la nube (Escalable casi Indefinidamente)	Límite físico según la capacidad del hardware
Mayor tolerancia a fallos	Un fallo puede afectar todo el sistema
Más compleja de administrar	Más simple pero con límites

¿Por qué Node.js favorece la escalabilidad horizontal?

- Node.js está diseñado para:
 - Ejecutar múltiples instancias ligeras
 - Integrarse fácilmente con balanceadores de carga
 - Encajar en arquitecturas de microservicios
 - Desplegarse fácilmente en entornos Cloud (AWS, Azure, GCP)
 - Replicar procesos en lugar de añadir hilos
- Node.js **no escala añadiendo hilos** dentro del mismo proceso
 - Escala replicando procesos.

¿Porqué la escalabilidad horizontal?

- **Mayor capacidad**
 - Permite distribuir la carga entre múltiples servidores
- **Alta disponibilidad**
 - Si un nodo falla, los demás continúan funcionando
- **Elasticidad en la nube (costes flexibles)**
 - Se pueden añadir o eliminar instancias según la demanda
- **Distribución geográfica**
 - Reduce latencia al desplegar servicios en distintas regiones

Características y Arquitectura de Node.js



Características y Arquitectura de Node.js

- Node.js se basa en tres pilares fundamentales:
 - Diseño modular
 - Modelo de operaciones I/O asíncrona y no bloqueante (Non-blocking)
 - Arquitectura basada en eventos (Single Threaded Event Loop)



Basado en un diseño
modular

Basado en un diseño modular

- Node.js está basado en módulos
 - Cada funcionalidad se divide en paquetes independientes
 - Permite reutilización y mantenimiento sencillo
 - Incluye módulos Core (http, fs, path, etc.)
 - Se pueden añadir módulos externos mediante NPM o Yarn

Basado en un diseño modular

- **NPM (Node Package Manager)**

- Es un **gestor de paquetes** para JavaScript y Node.js.
- **Facilita** a los desarrolladores de JavaScript **reutilizar el código** que otros desarrolladores han compartido.
- Hay más de 600,000 paquetes de código JavaScript disponibles para descargar.
- NPM está escrito en Node.js, por lo que su sistema necesita tener instalado Node.js
- En la **instalación de Node.js**, por defecto **se instala NPM**.
- Página oficial:
 - <https://docs.npmjs.com/>



Basado en un diseño modular

- **Instalación de módulos usando npm (II)**

- Los módulos se descargan e instalan **localmente** mediante el comando
 - Esto creará el directorio **node_modules** en el directorio actual (si no existe) y descargará el paquete a ese directorio.

```
npm install <package_name>
```



Basado en un diseño modular

- **Declarar el uso de un paquete en una aplicación**
 - Es necesario que el paquete este instalado previamente.
 - Incluir el paquete el fichero *package.json* de la aplicación.
 - Por defecto, cuando se instala un paquete se añade a este fichero como dependencia de la aplicación.



```
1 {
2   "name": "myvirtualstore",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "dependencies": {
13    "body-parser": "^1.19.0",
14    "express": "^4.17.1",
15    "swig": "^1.4.2"
16  }
17 }
```

Basado en un diseño modular

- **Usar un paquete en una aplicación Node.js (II)**

- En el fichero js correspondiente, se añade el paquete usando la siguiente sintaxis:
- Ejemplo:

```
var my_package = require('<package_name>')
```

```
var express = require('express');  
var app = express();
```



Basado en un diseño modular

- Algunos comandos importantes de npm

Comando	Descripción
<code>npm install <package_name></code> Ej. <code>npm install express</code>	Instala las dependencias en la carpeta local del proyecto node_modules <u>Por defecto instala la ultima versión de la dependencia</u>
<code>npm install <package_name> -g</code> Ej: <code>npm install grunt -g</code>	Instala las dependencias en el directorio de trabajo como un paquete global
<code>npm install <package_name> --no-save</code>	No agrega la declaración de la dependencia al package.json
<code>npm install <package_name> --save</code>	Agrega la dependencia al package.json
<code>npm install <name>@<version></code> Ej. <code>npm install express@4.16.2</code>	Instala la versión especificada de la dependencia
https://docs.npmjs.com/cli/install	



Modelo de I/O asíncrono y no
bloqueante

Modelo de I/O asíncrono y no bloqueante

- **¿Qué significa ?**

- Las operaciones de entrada/salida no bloquean el hilo principal
- El programa puede continuar ejecutándose

- **Node.js usa I/O asíncrono para tareas como:**

- Leer o escribir en el sistema de archivos
- Almacenar información en bases de datos
- Establecer comunicación de red o comunicarse con otros componentes

- **¿Qué ocurre internamente?**

- Estas operaciones se delegan al sistema operativo o al motor de base de datos
- Mientras tanto, el hilo principal no se bloquea y puede atender otras peticiones

- **Resultado**

- Permite gestionar múltiples operaciones de E/S de forma concurrente sin bloquear el sistema



Comunicación síncrona

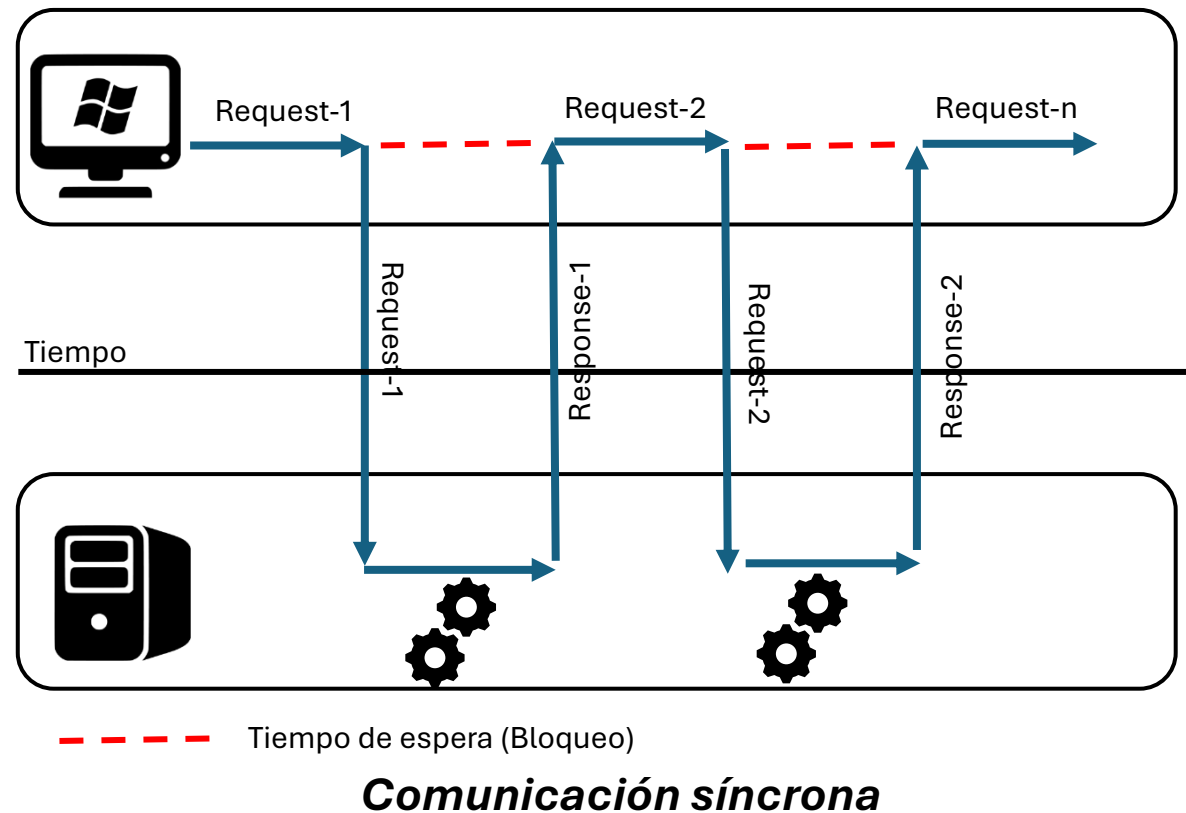
Comunicación síncrona

- **Características de una comunicación *síncrona***
 - Las **operaciones** se ejecutan de forma **secuencial**.
 - Las operaciones son **bloqueantes** (blocking).
 - El **programa** permanece **bloqueado** hasta que termine la operación.
 - El **cliente espera la respuesta del servidor para continuar** con el flujo del programa.
 - Son **menos eficientes, pero más simples y fiables** que las operaciones asíncronas.



Comunicación síncrona

- **Funcionamiento comunicación síncrona**



Comunicación síncrona

- Ejemplo comunicación síncrona en JavaScript

```
<script>
var ages = [8, 10, 15, 25, 65, 23, 18, 55, 75, 88, 77, 99, 100]
function Search(minAge, maxAge) {
  console.log('START search with age between:', minAge, "and", maxAge);
  result = ages.filter(function(age) {
    return age >= minAge && age <= maxAge;
  });
  return result;
}

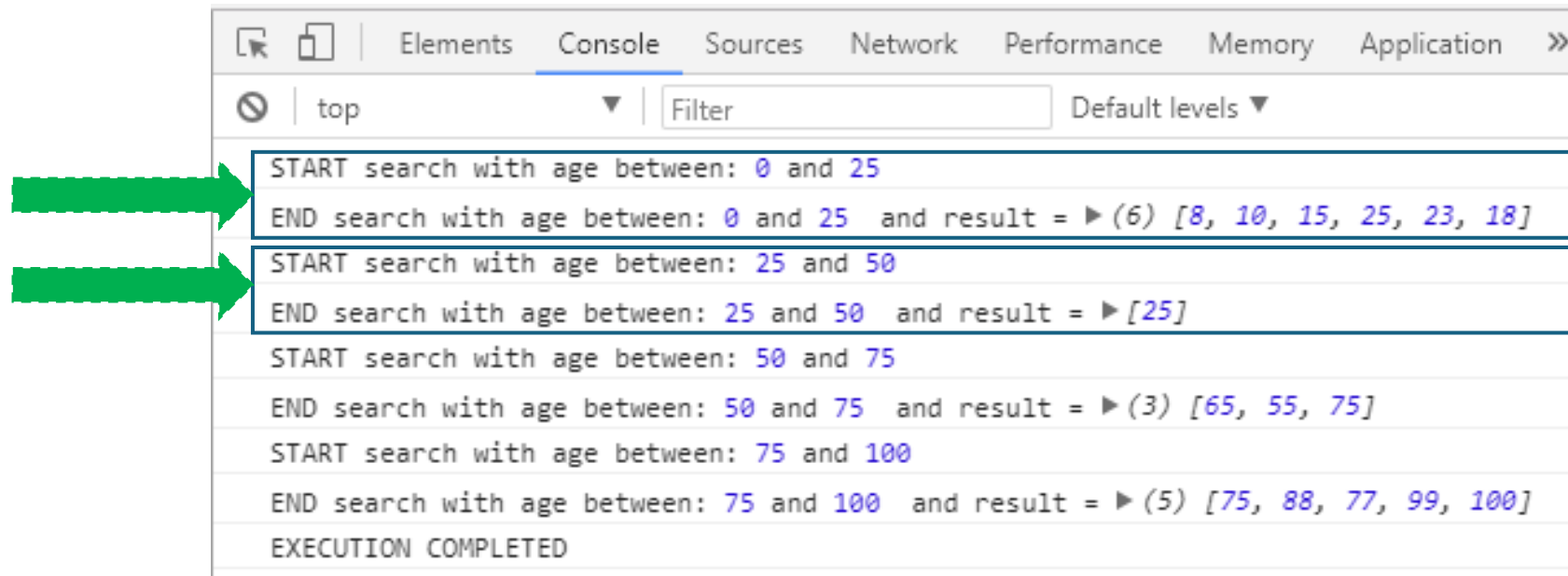
function executeSearch() {
  var maxAge = 0;
  var maxRange = 4;
  for (var i = 0; i < maxRange; i++) {
    minAge = maxAge;
    maxAge = maxAge + 25;
    result = Search(minAge, maxAge);
    console.log('END search with age between:', minAge, "and",
      maxAge, ' and result =', result);
  };

  console.log('EXECUTION COMPLETED');
}
</script>
```



Comunicación síncrona

- Ejemplo comunicación síncrona en JavaScript > Resultado



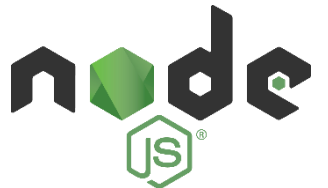
```
START search with age between: 0 and 25
END search with age between: 0 and 25 and result = ► (6) [8, 10, 15, 25, 23, 18]
START search with age between: 25 and 50
END search with age between: 25 and 50 and result = ► [25]
START search with age between: 50 and 75
END search with age between: 50 and 75 and result = ► (3) [65, 55, 75]
START search with age between: 75 and 100
END search with age between: 75 and 100 and result = ► (5) [75, 88, 77, 99, 100]
EXECUTION COMPLETED
```



Comunicación asíncrona

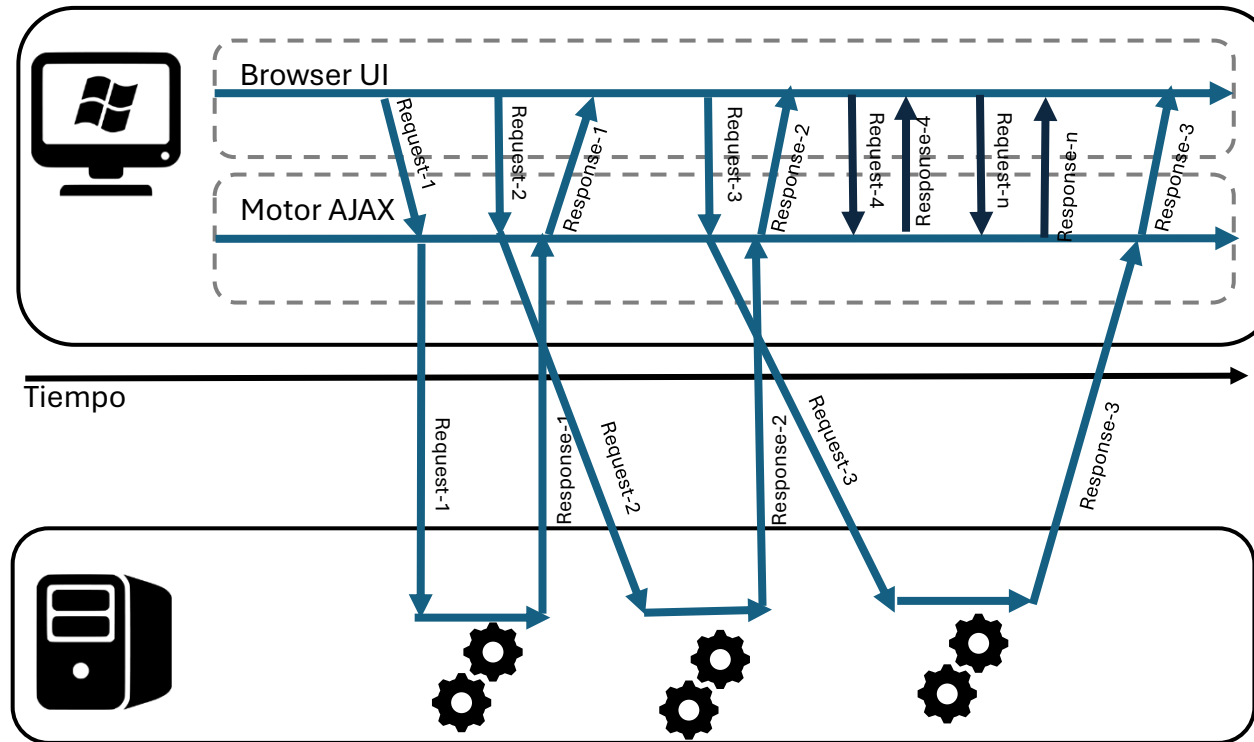
Comunicación asíncrona

- **Características de una comunicación *asíncrona***
 - Las operaciones **son NO bloqueantes**.
 - No se espera a que una operación termine para continuar con el flujo del programa
 - Normalmente se realizan mediante el sistema de **callback(retrollamadas), promesas o async/await**.
 - En Node.js, si un proceso de I/O tarda mucho tiempo, entonces permite que continúe otro proceso antes de que la transmisión haya finalizado.



Comunicación asíncrona

- **Funcionamiento comunicación asíncrona**



Comunicación asíncrona (Modelo AJAX)



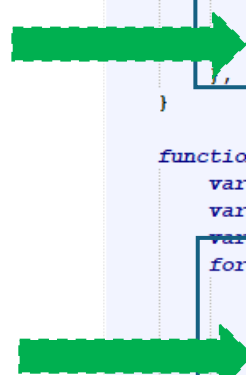
Comunicación asíncrona

- Ejemplo Comunicación asíncrona en JavaScript usando *callback*

```
<script>
var ages = [8, 10, 15, 25, 65, 23, 18, 55, 75, 88, 77, 99, 100]
function asyncSearch(minAge, maxAge, callback) {
  console.log('START search with age between:', minAge, "and", maxAge);
  setTimeout(function() {
    result = ages.filter(function(age) {
      return age >= minAge && age <= maxAge;
    });
    callback(minAge, maxAge, result);
  }, 0 | Math.random() * 2000);
}

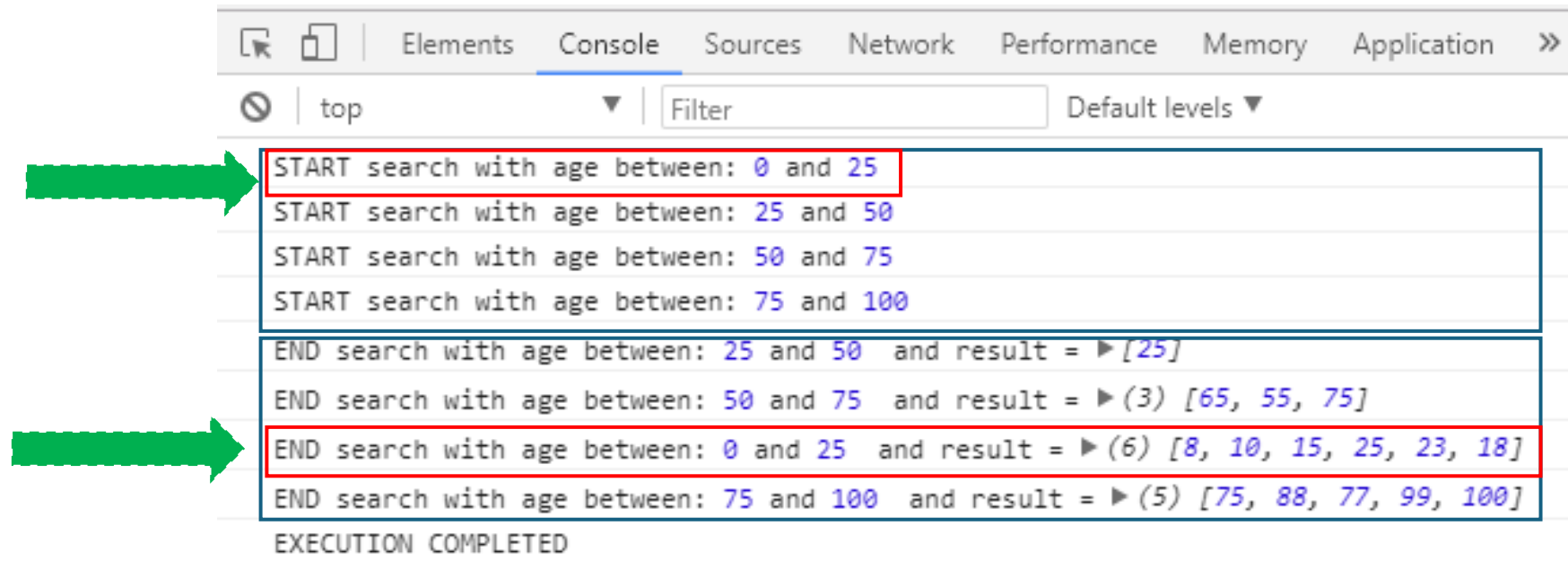
function executeAsyncSearch() {
  var maxAge = 0;
  var maxRange = 4;
  var count = 0;
  for (var i = 0; i < maxRange; i++) {
    minAge = maxAge;
    var maxAge = maxAge + 25;
    asyncSearch(minAge, maxAge, function (minAge, maxAge, result) {
      console.log('END search with age between:', minAge, "and", maxAge,
        ' and result =', result);

      if (++count === maxRange) {
        console.log('EXECUTION COMPLETED');
      }
    });
  }
}
</script>
```



Comunicación asíncrona

- Ejemplo Comunicación asíncrona en JavaScript usando *callback* > Resultado



```
START search with age between: 0 and 25
START search with age between: 25 and 50
START search with age between: 50 and 75
START search with age between: 75 and 100
END search with age between: 25 and 50 and result = ► [25]
END search with age between: 50 and 75 and result = ► (3) [65, 55, 75]
END search with age between: 0 and 25 and result = ► (6) [8, 10, 15, 25, 23, 18]
END search with age between: 75 and 100 and result = ► (5) [75, 88, 77, 99, 100]
EXECUTION COMPLETED
```



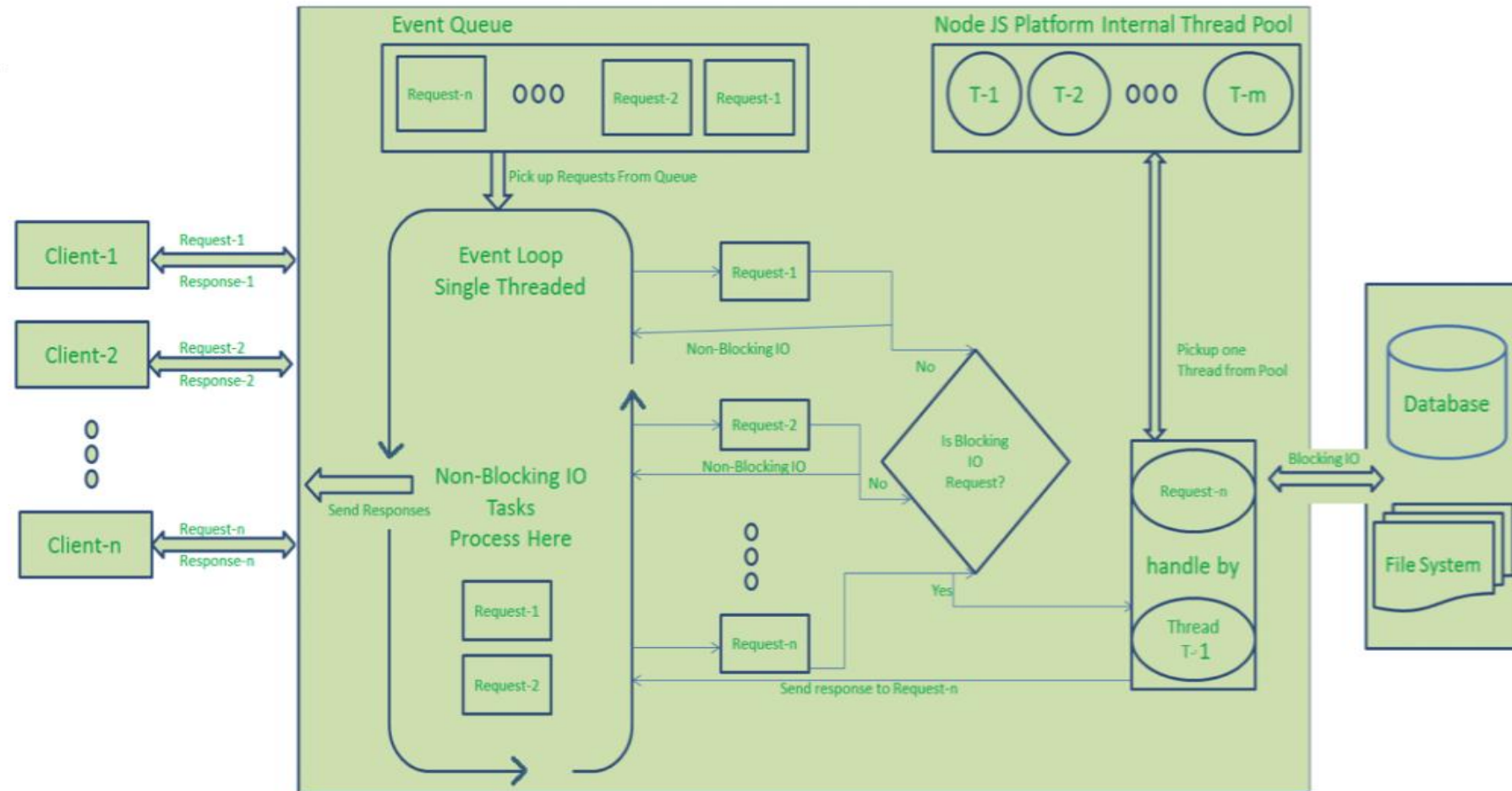
Arquitectura basada en eventos

Arquitectura basada en eventos

- Node.js se basa en la arquitectura “**Single Threaded Event Loop**”
 - Utiliza un **único hilo de ejecución** que gestiona las peticiones concurrentes de los clientes
 - Las peticiones generan eventos
 - El **Bucle de Eventos (Event Loop)** gestiona la cola de eventos
 - En cada petición realizada por un cliente, Node.js **no genera un nuevo hilo**, sino, que disparará un evento dentro del **Event Loop**.



Arquitectura Node.js



Node JS Application/Node JS Server
Fuente: <https://www.journaldev.com>



¿Por qué Node.js mejora la concurrencia?

- No crea un hilo por petición
- Reduce consumo de memoria
- Gestiona miles de conexiones simultáneamente

¿Cuándo usar Node.js?

- Es una buena opción cuando:
 - Se necesitan muchas conexiones concurrentes
 - La aplicación es intensiva en operaciones I/O
 - Se desarrollan APIs REST o microservicios
 - Se construyen aplicaciones en tiempo real (chat, streaming, colaboración)
 - Se trabaja en arquitecturas distribuidas o cloud

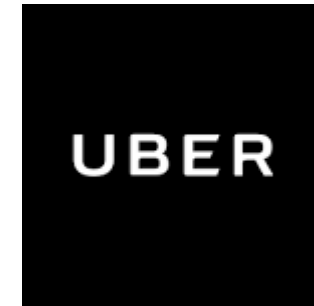


¿Cuándo usar NO Node.js?

- Puede no ser la mejor opción cuando:
 - La aplicación es intensiva en CPU
 - Se requiere procesamiento científico o matemático pesado
 - Se necesita paralelismo nativo complejo
 - La lógica de negocio es extremadamente computacional



¿Quiénes usan Node.js?



GitHub



Entorno de desarrollo

Entorno de desarrollo

- Requisitos:
 - Descargar e instalar Node.js en el sistema operativo.
- Las aplicaciones Node.js pueden desarrollarse en prácticamente cualquier **editor o IDE**
 - Notepad, Visual Studio Code, Eclipse, *IntelliJ IDEA*, *WebStorm*, Spring Tool Suite, etc.
 - Plugins requeridos (según el IDE)
 - *JavaScript and TypeScript*
 - *Node.js*



Entorno de desarrollo

- Descargar e instalar Node.js desde el sitio oficial:
 - <https://nodejs.org/es/download/>
- Versiones disponibles:
 - **LTS (Long Term Support)** → versión recomendada para producción.
 - **Current** → versión más reciente con las últimas funcionalidades.

Obtiene Node.js® v24.13.1 (LTS) para Windows usando Docker con npm

Info: Want new features sooner? Get the **latest Node.js version** instead and try the latest improvements!

```
1 # Docker provee instrucciones dedicadas para cada sistema operativo.
2 # Por favor consulta la documentación oficial en https://www.docker.com/get-started/
3
4 # Descarga la imagen de Docker de Node.js:
5 docker pull node:24-alpine
6
7 # Crea un contenedor de Node.js e inicia una sesión shell:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verifica la versión de Node.js:
11 node -v # Debería mostrar "v24.13.1".
12
13 # Verifica versión de npm:
14 npm -v # Debería mostrar "11.8.0".
```

PowerShell </> Copiar al portapapeles

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](https://www.docker.com)

O obtiene una versión pre compilada de Node.js® para Windows usando la arquitectura x64

[Windows Installer \(.msi\)](#) [Standalone Binary \(.zip\)](#)

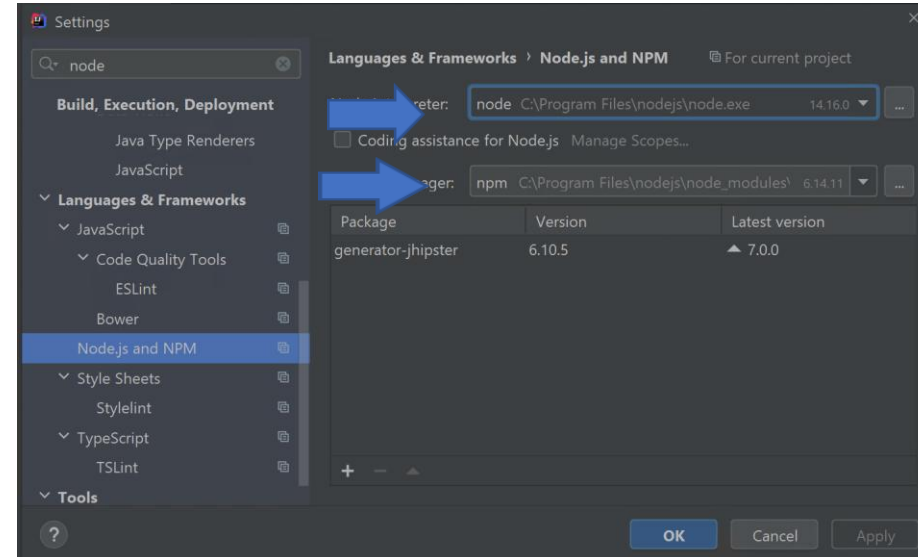


¿Qué versión de Node.js instalar?

Característica	Versión Current (Actual)	Versión LTS (Long-Term Support)
Propósito	Últimas características y mejoras experimentales	Estabilidad y soporte prolongado
Actualización	Cada 6 meses (abril y octubre)	Se mantiene durante 30 meses (18 meses de soporte activo + 12 meses de mantenimiento)
Estabilidad	Menos estable, puede incluir cambios experimentales	Alta estabilidad (probada y confiable)
Nuevas funcionalidades	Todas las últimas innovaciones y mejoras	Solo mejoras críticas y correcciones de seguridad
Uso recomendado	Desarrollo, pruebas y nuevas tecnologías	Producción y entornos críticos
Compatibilidad con paquetes	Puede tener problemas con paquetes antiguos	Mayor compatibilidad con paquetes populares
Ejemplo de versiones	Node.js 25.6.1 (Current)	Node.js 24.14.0 (LTS)

Entorno de desarrollo > IntelliJ IDEA

- En IntelliJ IDEA Ultimate, los plugins de JavaScript y Node.js vienen instalados y habilitados por defecto.
- IntelliJ permite trabajar con diferentes versiones de Node.js en un mismo proyecto
- Para gestionar múltiples versiones se puede utilizar:
 - Node Version Manager (NVM)
 - <https://github.com/nvm-sh/nvm>
- **NVM permite:**
 - Instalar diferentes versiones de Node.js
 - Cambiar fácilmente entre versiones

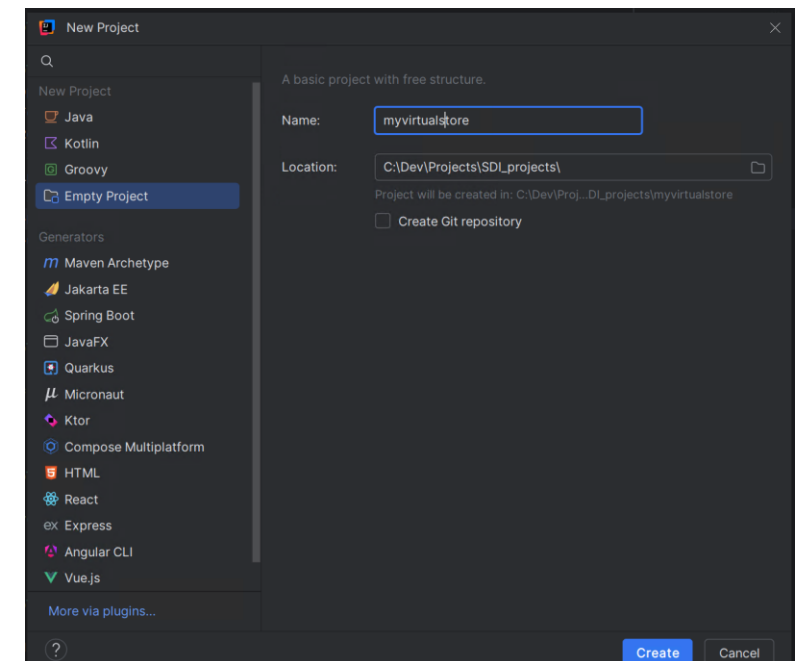


```
$ nvm use 16
Now using node v16.9.1 (npm v7.21.1)
$ node -v
v16.9.1
$ nvm use 14
Now using node v14.18.0 (npm v6.14.15)
```



Aplicación

- Ejemplo de creación de una aplicación web “Hola mundo” con Node.js en IntelliJ IDEA.
- Ruta de creación del proyecto:
 - Menú **File | New | Project | Empty Project**
- Primero se crea un proyecto vacío y, a partir de ahí, se incorporan los ficheros básicos de la aplicación.



Aplicación > Estructura básica

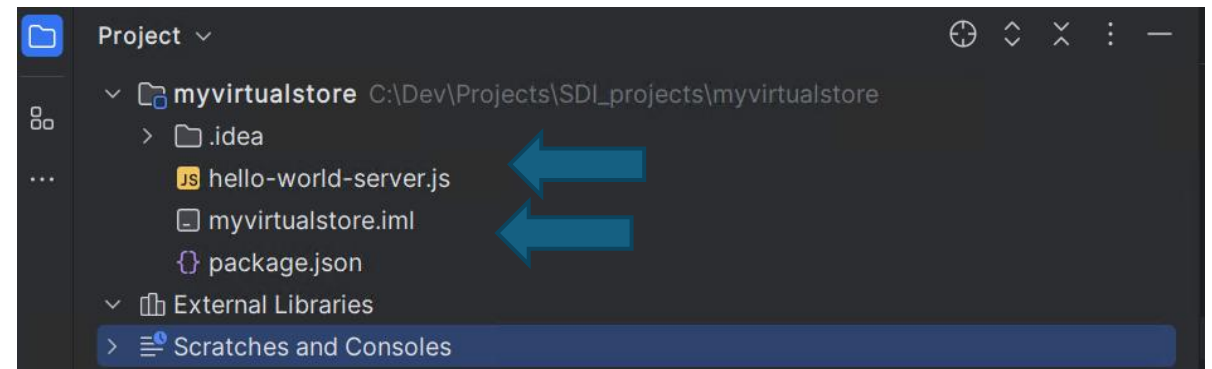
- Una **aplicación Node.js** sencilla **necesita** al menos **dos** **ficheros principales**:

1. **package.json** *que* define:

- La **configuración** de la aplicación
- Los **metadatos** del proyecto
- Las **dependencias** utilizadas

2. **Fichero JavaScript con la lógica del servidor**

- Ejemplo:
 - hello-world-server.js



Aplicación > Server

- El fichero `hello-world-server.js` contiene la lógica de la aplicación
- Inicialmente se define una variable que importa el módulo:
 - **http**
 - Este módulo forma parte del **Core de Node.js** y permite crear servidores web simples.
 - Sin embargo, en aplicaciones reales normalmente se utiliza **Express**, que facilita el desarrollo de aplicaciones web.

```
JS hello-world-server.js x
1 let http = require('http');
2 http.createServer(function handler(req : IncomingMessage , res : ServerResponse ) {
3   res.writeHead( statusCode: 200, headers: {'Content-Type': 'text/plain'});
4   res.end( chunk: 'Hello World\n');
5 }).listen( port: 1337, hostname: '127.0.0.1');
6 console.log('Server running at http://127.0.0.1:1337');
```



Aplicación > Gestión de dependencias

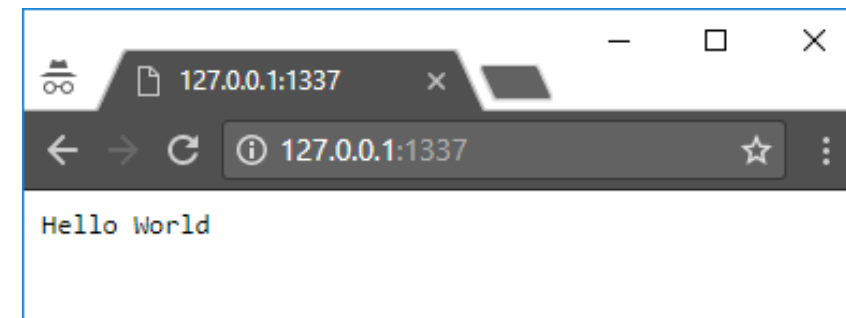
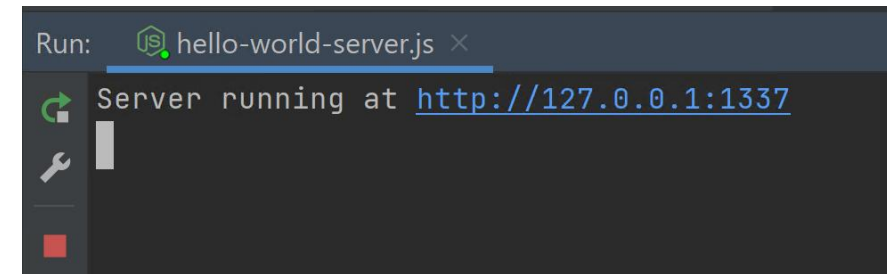
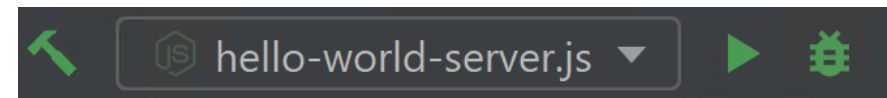
- En cada aplicación Node.js debe existir un archivo: **package.json** ubicado en la carpeta raíz de la aplicación.
- Este fichero define:
 - La **configuración** de la aplicación
 - Los **metadatos del proyecto**
 - Las **dependencias** utilizadas
- No es obligatorio especificar todas las dependencias, pero **sí es muy recomendable hacerlo**.
- Las dependencias se instalan utilizando:
 - npm (Node Package Manager)
 - Yarn
- Por defecto, las dependencias se añaden a la sección **dependencias** del fichero **package.json**

```
1 {
2   "name": "myvirtualstore",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "dependencies": {
13    "body-parser": "^1.19.0",
14    "express": "^4.17.1",
15    "swig": "^1.4.2"
16  }
17 }
```



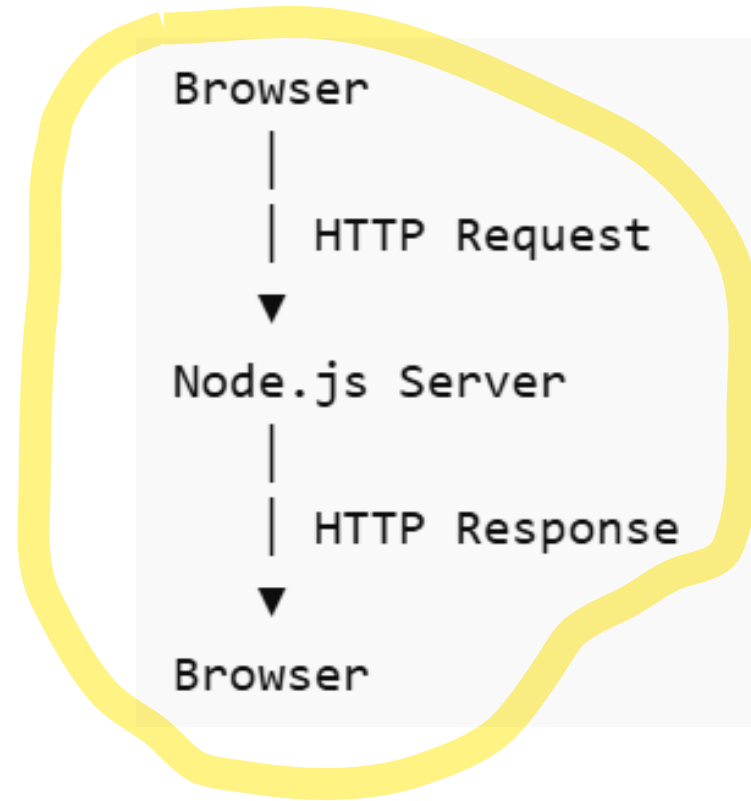
Aplicación > Despliegue

- Para ejecutar la aplicación:
 - Clic derecho en el fichero hello-world-server.js
 - Seleccionar| **Run hello-world-server.js...**
- En la consola se mostrará:
 - El estado de ejecución del servidor
 - Los mensajes generados mediante console.log
- Una vez iniciado el servidor, podemos acceder desde el navegador:
 - <http://127.0.0.1:1337/>



Flujo de ejecución de una aplicación Node.js

1. Se ejecuta el archivo JavaScript del servidor
 - `node hello-world-server.js`
2. Node.js inicia un servidor HTTP
3. El servidor queda escuchando peticiones en un puerto
 - <http://localhost:1337>
4. Cuando el navegador realiza una petición:
 - Cliente (Browser) → Servidor Node.js
5. El servidor procesa la petición y devuelve una respuesta.



Frameworks para Node.js

- Node.js proporciona las herramientas básicas para crear servidores y aplicaciones web.
- Sin embargo, para facilitar el desarrollo existen **frameworks** que ofrecen funcionalidades adicionales.
- Los frameworks ayudan a:
 - organizar el código
 - gestionar rutas y peticiones HTTP
 - desarrollar aplicaciones web y APIs de forma más sencilla.



Frameworks para Node.js



koa



Express



 **Total.js** Platform



METEOR

Comparativa de Frameworks Node.js

Característica	Express.js	Nest.js	AdonisJS	Fastify
Propósito	Framework minimalista para aplicaciones web y API REST	Framework progresivo basado en TypeScript con arquitectura modular	Framework completo con enfoque en estructura y desarrollo rápido	Framework de alto rendimiento, enfocado en rapidez y eficiencia
Arquitectura	Basado en middleware, sin estructura impuesta	Modular, basado en controladores y servicios. Orientada a componentes (similar a Angular)	MVC (Modelo-Vista-Controlador)	Basado en esquema de plugins y hooks para mayor flexibilidad
Enfoque	Minimalista y Flexibilidad	Escalabilidad, mantenibilidad	Productividad, convención sobre configuración	Velocidad, eficiencia
Curva de aprendizaje	Fácil para principiantes	Requiere conocimientos de TypeScript y programación modular	Más pronunciada debido a su estructura y características avanzadas	Similar a Express, pero con enfoque en configuración optimizada
Lenguaje	JavaScript, compatible con TypeScript	TypeScript (compatible con JavaScript)	TypeScript (compatible con JavaScript)	JavaScript, compatible con TypeScript
Manejo de Middleware	Basado en funciones app.use()	Basado en decoradores y módulos	Integrado con una capa de middleware robusta	Basado en hooks y esquema de plugins optimizados
Soporte para WebSockets	A través de paquetes externos, por ejemplo, socket.io y ws.	Integrado con soporte nativo	Integrado con soporte nativo	A través de paquetes externos
Rendimiento	Ligero y rápido	Ligero, pero más pesado que Express por su estructura. Mejora si con Fastify como base)	Más pesado debido a sus características avanzadas	Muy rápido, optimizado para baja latencia y alto rendimiento
Adecuado para	Aplicaciones pequeñas y medianas, APIs REST	Aplicaciones escalables, arquitecturas empresariales y microservicios	Aplicaciones empresariales full-stack, con estructura predefinida	APIs de alto rendimiento y microservicios
Comunidad	Muy grande y activa	Creciente y activa	Mediana pero dedicada	Creciente y activa

Express

- Express es un **framework de desarrollo web rápido y flexible para Node.js**.
- Es el framework **más utilizado en el ecosistema Node.js**
- Características principales:
 - **Routing:** manejo de URLs y rutas de la aplicación
 - **Gestión de peticiones HTTP:** GET, POST, PUT, DELETE, etc.
 - **Motores de plantillas:** integración con EJS, Pug (Jade), Twig, entre otros
 - **Configuración flexible:** permite definir puerto, middleware, vistas, etc.
- Express no impone una arquitectura estricta, lo que ofrece **gran flexibilidad**, aunque en proyectos grandes puede requerir una buena organización del código.



Express > Instalación

- El módulo **Express** no pertenece al **Core de Node.js**, por lo que debe instalarse mediante un gestor de paquetes.
- Módulos del Core de Node.js:
 - <https://nodejs.org/api/modules.html>
- Para instalar Express se utiliza normalmente NPM
- Comando de instalación:

```
npm install express
```

- Después de la instalación se crea la carpeta
 - **node_modules** que contiene las librerías instaladas en el proyecto.



Express > Express Generator

- **Express Generator**

- Herramienta de línea de comandos que permite *generar automáticamente la estructura básica de una aplicación Express*
- Permite crear rápidamente un proyecto con una estructura organizada.

- **Características principales:**

- **Crea la estructura inicial del proyecto:**
 - routes
 - views
 - public
- **Permite elegir un motor de plantillas como:**
 - Pug
 - EJS
 - Twig.
- **Genera archivos básicos para la configuración inicial.**
- **Incluye middleware básico**, como el manejo de errores y el soporte para archivos estáticos.



Express > Generador Apps

- Para crear una aplicación Express usando el generador:
 - Generar la aplicación con el comando npx (disponible a partir de Node.js 8.2.0).

```
npx express-generator --view=twig myapp2
```

- O instalar el módulo si tenemos una versión de Node.js anterior a la 8.2.0

```
npm install -g express-generator  
express --view= twig myapp2
```

- Instalar dependencia y ejecutar aplicación

```
$ cd myapp2  
$ npm install  
$ npm start
```



Express > Estructura Aplicación

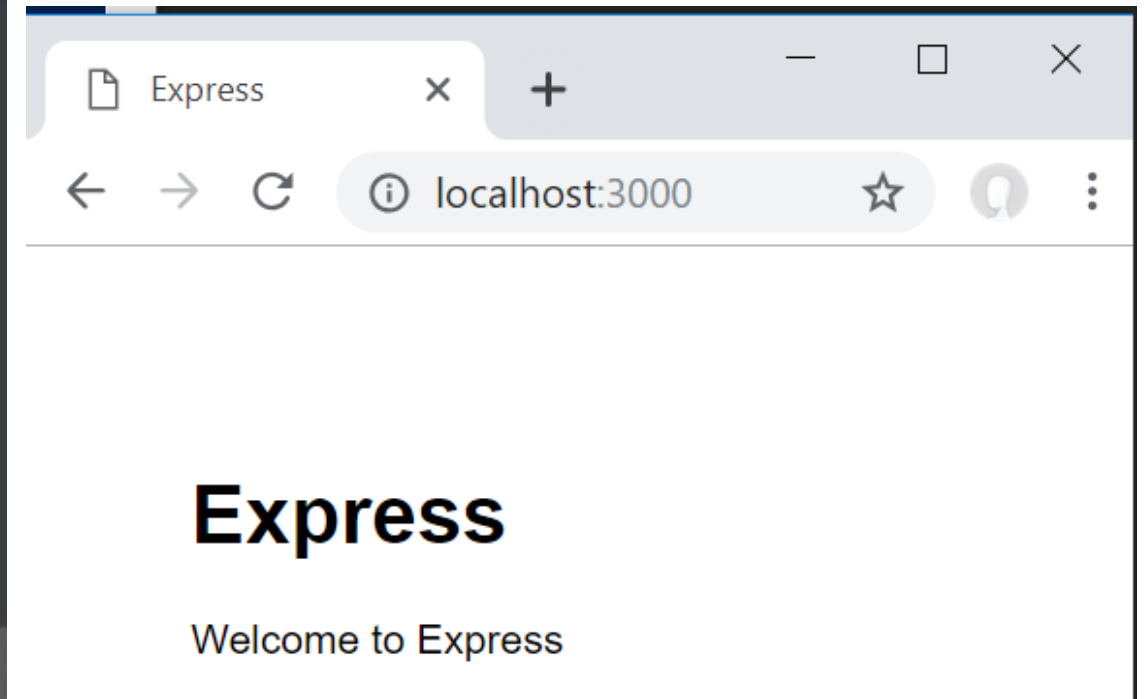
```
Terminal Local +
C:\Dev\Projects\S01_projects\sdiIDGIT-lab-nodejs>npx express-generator --view=twig musicstoreapp
npx: installed 10 in 3.469s

  create : musicstoreapp\
  create : musicstoreapp\public\
  create : musicstoreapp\public\javascripts\
  create : musicstoreapp\public\images\
  create : musicstoreapp\public\stylesheets\
  create : musicstoreapp\public\stylesheets\style.css
  create : musicstoreapp\routes\
  create : musicstoreapp\routes\index.js
  create : musicstoreapp\routes\users.js
  create : musicstoreapp\views\
  create : musicstoreapp\views\error.twig
  create : musicstoreapp\views\index.twig
  create : musicstoreapp\views\layout.twig
  create : musicstoreapp\app.js
  create : musicstoreapp\package.json
  create : musicstoreapp\bin\
  create : musicstoreapp\bin\www

  change directory:
    > cd musicstoreapp

  install dependencies:
    > npm install

  run the app:
    > SET DEBUG=musicstoreapp:* & npm start
```



Express > Estructura Aplicación

- **Estructura básica de una aplicación Express**
- **app.js:**
 - Fichero principal de configuración de la aplicación.
- **Routes**
 - Define las rutas (endpoints) de la aplicación.
- **Views**
 - Contiene las plantillas utilizadas para generar las páginas.
- **Public**
 - Archivos estáticos como:
 - Imágenes
 - CSS
 - JavaScript del cliente.
- **package.json**
 - Define la configuración del proyecto y sus dependencias

```
Application
├─ app.js
├─ bin
│   └─ www.js
├─ public
│   └─ images
│   └─ javascripts
│   └─ stylesheets
│       └─ style.css
├─ routes
│   └─ index.js
│   └─ users.js
└─ views
    └─ error.html
    └─ index.html
└─ package.json
```



Express > Aplicación

- **App.js**

- Es el **fichero principal** y punto de entrada de la aplicación Express.
- En este fichero se realiza la **configuración inicial** de la aplicación.
- Conceptos clave:
 - **require()**: Permite importar módulos necesarios en la aplicación.
 - **express()**: Crea una instancia de la aplicación Express.
 - **Middleware**: Funciones que procesan las peticiones HTTP antes de generar la respuesta.

Application

└─ app.js

...

```
let express = require('express');

let indexRouter = require('./routes/index');
let usersRouter = require('./routes/users');

let app = express();

// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'twig')

...

app.get('/songs', function (req, res) {
  res.send('lista de canciones');
});
```



Express > Aplicación

- **Carpeta bin**

- La carpeta bin contiene **archivos ejecutables o de arranque** de la aplicación.
- En una aplicación Express generada automáticamente suele incluir:
 - **bin/www.js**
- Este fichero se encarga
 - iniciar el servidor
 - definir el puerto de escucha
 - poner en marcha la aplicación Express.
- En este esquema:
 - **app.js** -> configura la aplicación
 - **bin/www.js** -> la ejecuta

Application

```
├─ bin
|   └─ www.js
... •
```

```
let app = require('..../app');

let http = require('http');

let port = normalizePort(process.env.PORT || '3000');

app.set('port', port);

let server = http.createServer(app);

server.listen(port);
server.on('listening', onListening);

...
```



Express > Aplicación

- **Uso de variables de aplicación**

- Permite declarar variables globales, que pueden ser usadas en cualquier parte de la aplicación
 - Por ejemplo: referencia al puerto, directorios, etc.
- ***app.set(clave, valor)***
 - Guarda una variable
 - Utiliza un string como clave
- ***app.get(clave)***
 - Recupera el valor de una variable
 - Utiliza un string como clave

Application

```
|— bin
|   |— www.js
...

```

```
...
let port = normalizePort(process.env.PORT || '3000');
app.set('port', port);

let server = http.createServer(app);

//server.listen(port);
server.listen(app.get('port'));
...
```



Express > Routing

- **Routing (enrutamiento o direccionamiento) en express**

- El **routing** define cómo responde el servidor cuando un cliente realiza una petición a una URL específica
- Define los **puntos finales (endpoints) de la aplicación**
- En Express, una **ruta** se define con la siguiente estructura:

`app.METHOD(PATH, HANDLER)`

Donde:

- **app**: es una instancia de una aplicación express.
- **METHOD**: método HTTP (GET, POST, PUT, DELETE, etc.)
- **PATH**: es una **ruta de acceso al servidor** o URL.
- **HANDLER**: es una función, matriz de funciones o un conjunto que procesa la petición y genera la respuesta.

```
let express = require('express');
let app = express();

...
app.get('/songs', function (req, res) {
  res.send('lista de canciones');
});

..
```



Express > Routing (métodos HTTP)

app.METHOD(PATH, HANDLER)

- En Express podemos definir rutas utilizando diferentes **métodos HTTP**.
- Cada **método HTTP** representa una operación diferente:
 - **GET** → obtener información
 - **POST** → crear un nuevo recurso
 - **PUT** → actualizar un recurso existente
 - **DELETE** → eliminar un recurso

```
let express = require('express');
let app = express();

...
app.get('/songs', function (req, res) {
  res.send('lista de canciones');
});

app.post('/songs/add', function (req, res) {
  res.send('lista de canciones');
});

app.put('/songs/:id', function (req, res) {
  res.send('lista de canciones');
});

..
```



Express > Routing (PATH)

app.METHOD(PATH, HANDLER)

- **Path (rutas)**

- En combinación con el método HTTP definen **los puntos finales (endpoints)**.
- Un **endpoint** es una URL específica donde los clientes pueden acceder a un recurso del servidor.
- Las rutas de acceso pueden ser:
 - Cadenas(string)
 - Patrones de cadenas
 - Expresiones regulares
- Ejemplos de **cadenas**:
 - El primer ejemplo responde a una solicitud GET en la pagina inicial de la aplicación usando la cadena “/”.
 - Los siguientes ejemplos responden en la ruta “/songs” a las peticiones GET, POST o PUT

```
let express = require('express');
let app = express();
...

app.get('/', function (req, res) {
  res.send(' ver página de inicio');
});

app.get('/songs/:id', function (req, res) {
  res.send('Info canción');
});

app.post('/songs', function (req, res) {
  res.send('lista de canciones');
});

app.put('/songs', function (req, res) {
  res.send('lista de canciones');
});
..
```



Express > Routing (PATH)

- **Enrutamiento y uso de comodines**

- En Express, las rutas pueden incluir **comodines y expresiones regulares** para hacerlas más flexibles
- Esto permite que una misma ruta pueda coincidir con **varias URLs diferentes**.
- Se pueden usar caracteres especiales como; **?, +, *, ()** y otros patrones de expresiones regulares

- **Ejemplo de uso de comodines**

- Uso de asterisco (*) → Coincide con cualquier cadena de caracteres después de un prefijo.

```
// /promo, /promocion, /promocionar
app.get('/promo*', function (req, res) {
  res.send('Respuesta al patrón promo*');
});
```

```
// /proclamar, /promOCIONar /procesar
app.get('/pro*ar', function (req, res) {
  res.send('Respuesta al patrón pro*ar');
});
```

- Uso del símbolo (+) → Indica que el carácter anterior debe aparecer **una o más veces**.

```
// ruta que coincide con abcd, abbcd, abbbcd, etc.
app.get('/ab+cd', function (req, res) {
  res.send('Respuesta al patrón ab+cd ');
});
```



Express > Routing (PATH)

- Ejemplo de uso de comodines
 - **Signo de interrogación (?)** → Indica que el carácter anterior es opcional.

```
// ruta que coincide Coincide con "color" y "colour".  
app.get('/colou?r', function (req, res) {  
  res.send('Respuesta al patrón colou?r');  
});
```

- **Expresiones regulares** → Se pueden definir rutas más avanzadas.
 - En Express, las rutas con expresiones regulares **no se definen como cadenas** entre comillas. Deben escribirse directamente como una expresión regular.

```
// Coincide con "/user/123", pero no con "/user/abc"  
app.get(/^\/user\/[0-9]+$/, function (req, res) {  
  res.send('Respuesta al patrón expresión regular /^\/user\/[0-9]+$/ ');  
});
```

<https://expressjs.com/es/guide/routing.html>



Express > Routing (PATH)

- **Ventajas** de usar comodines y patrones en rutar
 - **Mayor flexibilidad** al definir rutas.
 - **Reducir el número de rutas** necesarias en la aplicación.
 - **Gestionar URLs dinámicas** de forma más sencilla.
 - **Mejorar la organización del código.**
- Los comodines permiten que ***una sola ruta pueda responder a múltiples URLs similares.***



Express > Routing (HANDLER)

*app.METHOD(PATH, **HANDLER**)*

- Un **handler** es la función que se ejecuta cuando el servidor recibe una petición en una ruta específica.
- Puede ser:
 - una función
 - una matriz de funciones
 - una combinación de ambas
- Los handlers tienen acceso a:
 - **req (request)** → información de la petición del cliente
 - **res (response)** → objeto utilizado para enviar la respuesta al cliente

```
app.get('/songs', function (req, res) {  
  res.send('lista de canciones');  
});
```

```
let authenticator = function (req, res, next) {  
  console.log('ejecutar un autenticador');  
  next();  
}  
  
let validator = function (req, res, next) {  
  console.log('Por ejemplo, lógica de validación de datos');  
  datos_validos = true;  
  if (datos_validos == true) {  
    next();  
  } else {  
    res.send("Datos no válidos")  
  }  
}  
  
app.post('/songs', [authenticator, validator], function (req, res, next) {  
  console.log('Ejecutar aquí alguna logica de negocio');  
  next();  
}, function (req, res) {  
  res.send('Ver info canción');  
});
```

Express > Routing (HANDLER)

- Métodos de respuesta (Objeto response)
 - El objeto **response (res)** permite enviar la respuesta al cliente.
 - Los métodos de respuesta:
 - envían datos al cliente
 - finalizan el ciclo **petición-respuesta**
 - Hay que invocarlo desde un manejador de rutas(handler).
 - Si no se invoca un método de respuesta, **la petición quedará pendiente** y se producirá un timeout.

```
app.post('/song', function (req, res) {  
  res.send('lista de canciones');  
});
```

Cliente
|
HTTP Request
|
Handler
|
res.send()
|
HTTP Response



Express > Routing (HANDLER)

- **Métodos de respuesta (Objeto Response)**

- Hay que indicar uno de estos métodos para que una solicitud no quede pendiente.

Método	Descripción
<code>res.send()</code>	Envía una respuesta en forma de cadena
<code>res.json()</code>	Envía una respuesta en formato JSON.
<code>res.redirect()</code>	Redirecciona a otra URL. Ej redireccionar a “/home” <code>res.redirect("/home") ;</code>
<code>res.render()</code>	Renderiza una plantilla y envía la renderización como respuesta.
<code>res.sendFile()</code>	Envía un archivo como una secuencia de octetos o byte.
<code>res.sendStatus()</code>	Establece el código de estado de la respuesta y envía su representación de string como el cuerpo de respuesta.
Otros...	



Express > Routing (módulos)

- Las aplicaciones deben optar por **un diseño modular** para mejorar la arquitectura y facilitar la reutilización del código.
- Cada módulo se **encarga de gestionar las rutas relacionadas** con una entidad específica, por ejemplo:
 - usuarios
 - canciones
 - productos
 - etc.
- Los módulos de rutas se almacenan normalmente en la **carpeta routes**
- Estos módulos actúan como controladores y se encargan de:
 - definir los endpoints de la aplicación
 - gestionar las peticiones HTTP (GET, POST, PUT, DELETE)
 - delegar la lógica de negocio a otros componentes.

```
Application
├── app.js
├── bin
│   ├── www.js
├── package.json
├── public
│   ├── images
│   ├── javascripts
│   └── stylesheets
│       └── style.css
├── routes
│   ├── index.js
│   └── users.js
└── views
    ├── error.html
    └── index.html
```



¿Qué son los módulos?

- Los **módulos** permiten dividir el código de una aplicación en partes **reutilizables y organizadas**.
- En Node.js se utilizan para **separar funcionalidades** como:
 - Rutas
 - Controladores
 - Servicios
 - utilidades
- Node.js soporta dos sistemas de módulos:
 - **CommonJS (CJS) y ECMAScript Modules (ESM).**
- CommonJS (CJS)
 - Sistema tradicional de Node.js.
- **Sintaxis**
 - utiliza **require()** para importar módulos y **module.exports** para exportarlos.
- **Características:**
 - carga síncrona
 - ampliamente utilizado en proyectos Node.js.



¿Qué son los módulos?

- **CommonJS (CJS)**

- **Declarar módulo:**

- Para declarar o exportar un módulo en Node.js, se utiliza **module.exports**.
 - Esto permite exportar funciones, objetos o clases para su uso en otros archivos.

- **Parámetros en el módulo:**

- **Un módulo puede recibir parámetros** a través de su función o constructor, lo que permite personalizar su comportamiento.
 - Ejemplo común: Pasar una instancia de la aplicación.

- **Importar un módulo**

- Para incluir o importar un módulo en la aplicación, se utiliza **require(fichero)**.
 - Si el módulo recibe parámetros, estos se pasan al llamar la función o al crear una instancia.

```
module.exports = function (app) {  
  app.get('/songs', function (req, res) {  
    res.send('lista de canciones');  
  });  
}
```

```
// Dos formas distintas
```

```
require('./routes/songs')(app); // (app, param1, param2, etc.)  
let usersRouter = require('./routes/users');  
app.use('/users', usersRouter);
```



¿Qué son los módulos?

- **ESM (ECMAScript Modules)**

- Sistema moderno basado en el estándar de JavaScript.
- **Sintaxis**
 - Utiliza **import y export**, alineándose con los estándares modernos de JavaScript
- **Ejecución**
 - **Asíncrona (opcional):** Las importaciones pueden realizarse de forma asíncrona, lo que permite:
 - división del código (code splitting)
 - tiempos de carga iniciales más rápidos
- **Compatibilidad**
 - Introducido en Node.js 12.
 - Estable desde Node.js 14

```
// exportar el módulo de usuarios
const usersRouter = (app) => {
  app.get('/users', function (req, res) {
    res.send('lista de usuarios');
  });
};
export default usersRouter;
```

```
// Importar el módulo de usuarios
import usersRouter from './routes/users.js';
// Pasar la instancia de la app a la función
usersRouter(app);
```



Express > Routing (route())

- **app.route():** rutas encadenables en Express

- La función **app.route()** permite definir *varios métodos HTTP para una misma ruta.*
- *Evita la redundancia* al definir la ruta en una sola ubicación.
- Reduce errores tipográficos y facilita la organización del código.

```
module.exports = function (app) {  
  app.get("/songs", function (req, res) {  
    res.send("ver canciones");  
  });  
  
  app.post("/songs", function (req, res) {  
    res.send("crear una canción");  
  });  
};
```

```
module.exports = function (app) {  
  app.route('/songs')  
    .get(function (req, res) {  
      res.send("ver canciones");  
    })  
    .post(function (req, res) {  
      res.send("crear una canción");  
    })  
    .put(function (req, res) {  
      res.send("actualizar una canción");  
    })  
};
```

Express > Routing(route())

- Diferencia entre ***app.route()*** y ***app.use()***
 - ***app.route()***
 - Se utiliza para definir **múltiples métodos HTTP** para una misma ruta.
 - ***app.use()***
 - se usa para **registrar middleware** y aplicar lógica a múltiples rutas.

```
app.route('/songs')  
  .get(function (req, res) { res.send("ver canciones"); })  
  .post(function (req, res) { res.send("crear una  
canción"); })  
  
let bodyParser = require('body-parser');  
app.use(bodyParser.json());
```



Express > Peticiones web

- **Peticiones HTTP GET y parámetros**

- Las peticiones **GET** pueden incluir **parámetros en la URL** para enviar información al servidor.
- Existen **dos formas principales** de enviar parámetros:

1. **Parámetros en la query string:** Se envían como pares **clave y el valor** en la URL:
 - **?** inicia la lista de parámetros
 - `http://localhost/songs?name=despacito`
 - **&** separa múltiples parámetros
 - `http://localhost/songs?name=despacito&autor=Luis Fonsi`
2. **Parámetros en la ruta (Route Params):** El **valor del parámetro se incrusta** dentro de la URL sin especificar la clave.
 - `http://localhost/song/234/`



Express > Peticiones web

- Obtener parámetros de la Query String
 - En Express.js, los parámetros enviados en una petición GET mediante la query string se accede usando:
 - `req.query.<clave_parámetro>`
 - Si el cliente no envía el parámetro, el valor retornado será `"undefined"`
 - Buenas prácticas:
 1. Comprobar si el parámetro existe (null o undefined).
 2. Comprobar explícitamente si es undefined
 3. Usando valores por defecto (??)
 - Forma más moderna en JavaScript

```
module.exports = function (app) {  
  app.route('/songs')  
    .get(function (req, res) {  
      let response = "";  
      if (req.query.name != null)  
        response += 'Name: ' + req.query.name;  
      if (typeof (req.query.author) !== "undefined")  
        response += 'Author: ' + req.query.author;  
      res.send(response);  
    })  
    ...  
};
```

- *Usando valores por defecto (??)*

```
app.route('/songs')  
  .get(function (req, res) {  
    const name = req.query.name ?? "unknown";  
    const author = req.query.author ?? "unknown";  
    res.send(`Name: ${name} Author: ${author}`);  
  });
```



Express > Peticiones web

- **Obtener parámetros de la Query String**

- **Todos los valores** que obtenemos a través del **req.query** *son cadenas de texto*.
 - Sí queremos otro tipo de datos **debemos convertirlos**
 - Por ejemplo, para convertir en enteros usaríamos: `parseInt(req.query.num1)`
- JavaScript define varias funciones para cambiar el tipo de las variables, ejemplos:
 - `parseInt("valor")`
 - `parseFloat("valor")`

```
parseFloat("3.14");  
parseFloat("314e-2");  
parseFloat("0.0314E+2");
```



Express > Peticiones web

- **Obtener parámetros de ruta (Route Params):**

- Los **parámetros de ruta** son valores **incrustados en la URL** que permiten identificar recursos específicos
- En Express.js, estos parámetros:
 - Se definen en la ruta (path)
 - Se acceden mediante **req.params**
- Ejemplo de URL:
 - `http://localhost:8081/songs/121/`

```
app.get('/songs/:id', function (req, res) {  
  let response = 'Id: ' + req.params.id;  
  res.send(response);  
});
```



Parámetros en Express: req.query vs req.params

Tipo	Ejemplo URL	Acceso en Express	Uso típico
Query Params	/songs?author=LuisFonsi	<code>req.query.author</code>	filtros o búsquedas
Route Params	/songs/121	<code>req.params.id</code>	identificar un recurso



Express > Peticiones web

- Peticiones **HTTP POST y parámetros**

- Las peticiones **POST** se utilizan para **enviar información al servidor**, por ejemplo desde un formulario..
- A diferencia de las peticiones **GET**, los datos se envían en el **cuerpo de la petición (body)**.
- Los datos enviados pueden ser:
 - pares de clave-valor
 - texto plano
 - json
 - datos binario
 - etc.



Express > Peticiones web

- **Enviar parámetros en peticiones POST**

- En las peticiones **POST**, los parámetros se envían en el **cuerpo de la petición (body)**.
 - Por ejemplo, formulario que envía una petición **POST /songs**.
 - Define inputs con atributo **name**.
 - El **name** será la clave del parámetro, por ejemplo: nombre, genero, precio, etc.

```
<form method="post" action="/songs">  
  <input type="text" name="title" />  
  <input type="text" name="kind" />  
  <input type="number" name="price" />  
  
  ...  
</form>
```

Agregar canción

Nombre:

Genero:

Precio (€):

Imagen portada:
 Ningún archivo seleccionado

Fichero audio:
 Ningún archivo seleccionado



Express > Peticiones web

• Obtener parámetros en peticiones POST

- Para acceder a ellos en Express.js, es necesario usar *middleware* que procese el body de la petición, como body-parser o express.json()

- Para usar body-parser hay que instalar el módulo:

```
npm install body-parser
```

- Implementación

- Importar el módulo en la aplicación en *app.js*
- Añadir el middleware a la aplicación con *app.use*
- Se agregan funciones de parseo a la aplicación con *app.use()*

- *Json()*: Procesa cuerpos en **formato JSON**, muy común en APIs RES

```
{ "name": "despacito", "author": "Luis Fonsi" }
```

- *Urlencoded*: Procesa cuerpos enviados desde **formularios HTML**

extended: true: permite procesar estructuras de datos más complejas (objetos y array).

```
let app = express();
let bodyParser = require('body-parser');
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));
```



Express > Peticiones web

- **Obtener parámetros en peticiones POST**

- En Express.js, estos parámetros se obtienen mediante el objeto **body** del objeto de petición (**req**).
 - `req.body.<clave_parámetro>`
- Al igual que en parámetros anteriores estos podrían tomar valores **undefined** o **null**.

```
app.post('/songs', (req, res) => {  
  const title = req.body.title;  
  const kind = req.body.kind;  
  res.send(`Título: ${title}, Genero: ${kind}`);  
});
```



Express > Recursos estáticos

- Los **recursos estáticos** son archivos que el servidor envía al cliente **sin modificarlos**.
- ejemplos:
 - imágenes
 - archivos CSS
 - archivos JavaScript del lado del cliente
 - documentos HTML
- Express provee una función de asistencia (middleware) para facilitar el acceso a recursos estáticos.
- En aplicaciones Express, estos recursos suelen almacenarse en la carpeta: **public**.
- Express permite publicar una carpeta para que sus archivos puedan ser accedidos directamente desde el navegador.

```
Application
├─ app.js
├─ package.json
├─ public
│   └─ images
│   └─ javascripts
│   └─ stylesheets
│       └─ style.css
├─ routes
│   └─ index.js
│   └─ users.js
└─ views
    └─ error.html
    └─ index.html
```



Express > Recursos estáticos

- Para servir archivos estáticos en Express se configura la aplicación de esta forma:
 - **Caso 1:** Declarar un directorio estático estándar usando la función:
 - `express.static('<ruta del directorio>')`.
 - Los ficheros se obtienen desde la raíz del sitio '/'
 - **Caso 2:** Declarar un directorio estático con una **vía de acceso virtual** (donde la ruta NO existe realmente en el directorio de archivos), se tiene que crear un alias o prefijo.
 - `express.static('alias', '<ruta del directorio>')`.

```
├─ public
│   └─ images
│       └─ stylesheets
```

```
let app = express();
// Caso 1.
app.use(express.static('public'));
// acceso.

http://localhost:3000/images/user.png
http://localhost:3000/stylesheets/style.css

// Caso 2.
app.use('static', express.static('public'));
// acceso.

http://localhost:3000/static/images/user.png
http://localhost:3000/static/stylesheets/style.css
```

Motores de plantillas



Motores de plantillas

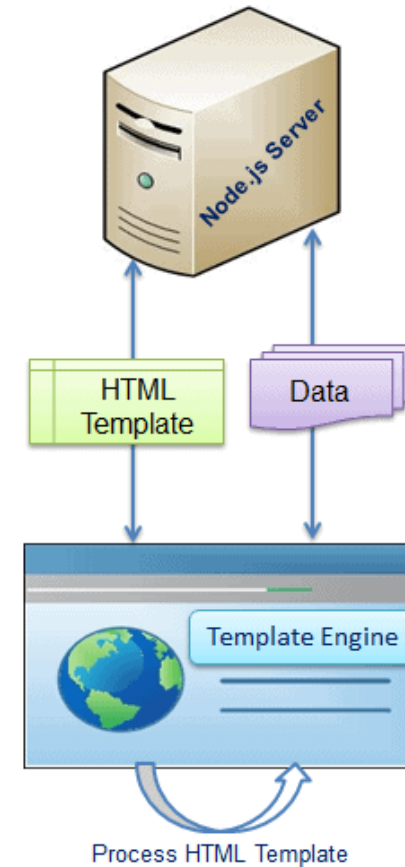
- Un **motor de plantillas** permite generar páginas **HTML dinámicas** utilizando datos enviados desde el servidor.
- Los motores de plantillas facilitan la **separación entre la interfaz de usuario y los datos (contenido)** de una aplicación web.
- Permiten procesar **datos del modelo** y convertirlos en **HTML dinámico** de forma automática.
- Tienen acceso a los **atributos del modelo**, pudiendo:
 - Insertarlos en el código HTML o JavaScript.
 - Utilizarlos en estructuras de control (condicionales, bucles, etc.)

- Algunos motores de plantillas populares son:



Motores de plantillas

- Un **sistema de plantillas** genera documentos dinámicos *combinando datos y una plantilla.*
- Está compuesto por los siguientes elementos:
 - **Modelo de datos**
 - Conjunto de datos que se utilizarán para generar el contenido dinámico.
 - **Plantilla(s)**
 - Archivo(s) que definen la **estructura del documento** (HTML, texto, etc.).
 - **Motor de plantillas**
 - Software que **combina los datos con la plantilla** para producir el resultado.
 - **Documento generado**
 - Resultado final generado dinámicamente.



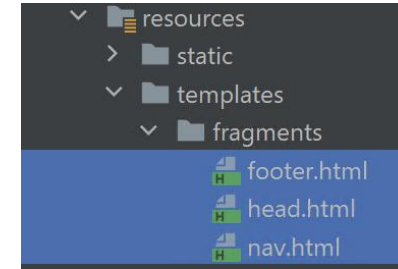
¿Por qué usar un motor de plantillas?

- Generar **contenido dinámico**.
- Separar **lógica de la aplicación y presentación**.
- Reutilizar **componentes de interfaz**.
- facilitar el mantenimiento del código.
- Ofrecen sistema de herencia y redefinición de bloques, permiten componer plantillas a partir de otras
 - Evitan replicar partes comunes en varias vistas (cabeceras, menús, pie de página, etc.).
- El mismo motor de plantillas puede ser **utilizado en diferentes frameworks**.



Motores de plantillas

- Además, los motores de plantillas suelen ofrecer mecanismos de composición de plantillas, como:
 - herencia de plantillas
 - reutilización de componentes comunes.



```
<!-- footer.html -->
<footer class="footer">
  <div class="container">
    <span class="text-white bg-primary">&copy; SDI - Gestión de notas</span>
  </div>
</footer>
```

```
<!DOCTYPE html SYSTEM "http://www.thymeleaf.org/dtd/xhtml1-strict-thymeleaf-spring4-4.dtd">
<html lang="en" xmlns="http://www.w3.org/1999/xhtml" xmlns:th="http://www.thymeleaf.org">
<head th:replace="fragments/head"></head>
<body>
  <nav th:replace="fragments/nav"></nav>
  <div class="container" style="text-align: center">
    <h2>Bienvenidos a la página principal</h2>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing</p>
  </div>
  <footer th:replace="fragments/footer"></footer>
</body>
</html>
```

Ejemplo de código usando Thymeleaf



Express > Vistas y plantillas

- Comúnmente las plantillas se almacenan en el directorio **views**
- La mayor parte de motores de plantillas deben ser instalados, por ejemplo, **Twig**
- Cuando usamos el generador de express ya incluye el motor de plantilla indicado en la generación, si no, podemos instalarlo con npm:

```
npm install twig
```



```
Application
├─ app.js
├─ package.json
├─ public
│   └─ images
│   └─ javascripts
│   └─ stylesheets
│       └─ style.css
├─ routes
│   └─ index.js
│   └─ users.js
└─ views
    └─ error.html
    └─ index.html
```



Express > Vistas y plantillas

- **Plantillas (atributos)**

- Los atributos del modelo se referencian con la siguiente sintaxis:
{{ <nombre_del_atributo> }}
- Ejemplo:
 - al renderizar la plantilla se insertará el valor del atributo **seller (vendedor)**

```
<html>
  <head>
    <title>Canciones</title>
  </head>
  <body>
    <h1> {{ seller }} </h1>
  </body>
</html>
```

Plantilla twig

```
{ "seller" : "uniovi" }
```

Atributos del modelo



Express > Vistas y plantillas

- **Plantillas (objetos)**

- Los valores de los atributos pueden ser también tipos **objetos**
- El operador `.` permite acceder a los atributos

```
<html>
  <head>
    <title>Canciones</title>
  </head>
  <body>
    <h1>{{ seller.name }}</h1>
  </body>
</html>
```

Plantilla twig

```
{ "seller" :
  {
    "name" : "uniovi",
    "year" : 2000
  }
}
```

Atributos del modelo



Express > Vistas y plantillas

• Plantillas (bucles)

- Los valores de los atributos pueden ser **colecciones**
- Ofrece **estructuras de control** para recorrer las colecciones
 - {% for <variable temporal> in <colección atributo del modelo> %} {% endfor %}

```
<body>
  <ul>
    {% for song in songs %}
      <li>
        {{ song.name }} -
        {{ song.price }}
      </li>
    {% endfor %}
  </ul>
</body>
```

Plantilla twig

```
{ "songs": [
  { "name" : "Blank space",
    "price" : "1.2" },
  { "name" : "See you again",
    "price" : "1.3" },
  { "name" : "Uptown Funk",
    "price" : "1.1" }
]}
```

Atributos del modelo



Express > Vistas y plantillas

- **Plantillas (estructuras condicionales)**

- Ofrece **estructuras condicionales** para incluir código en base a una expresión lógica, la sintaxis es la siguiente:
 - `{% if <expresión lógica> %} {% endif %}`

```
<body>
  {% if coche.precio < 100 %}
    <p> oferta </p>
  {% endif %}
</body>
```

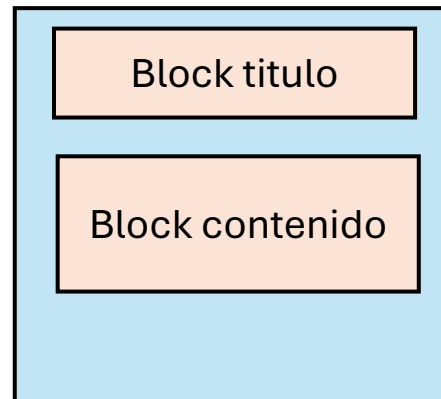
- Soporta casi el mismo conjunto de expresiones lógicas definido en Javascript
- Documentación completa:
<https://www.npmjs.com/package/twig>



Express > Vistas y plantillas

- **Composición de plantillas**

- Ofrece un sistema de **composición de plantillas** basado en **herencia** y redefinición de **bloques**
- Se define una **plantilla base** con todos los elementos comunes a todas las vistas
 - ejemplo: base.html
 - Se divide de forma lógica el código de la plantilla base en **bloques que podrán ser redefinidos en sus hijos**. (Ejemplos de bloques: cabeceras, título, contenido, etc.)
 - `{% block <nombre> %}` contenido redefinible `{% endblock %}`



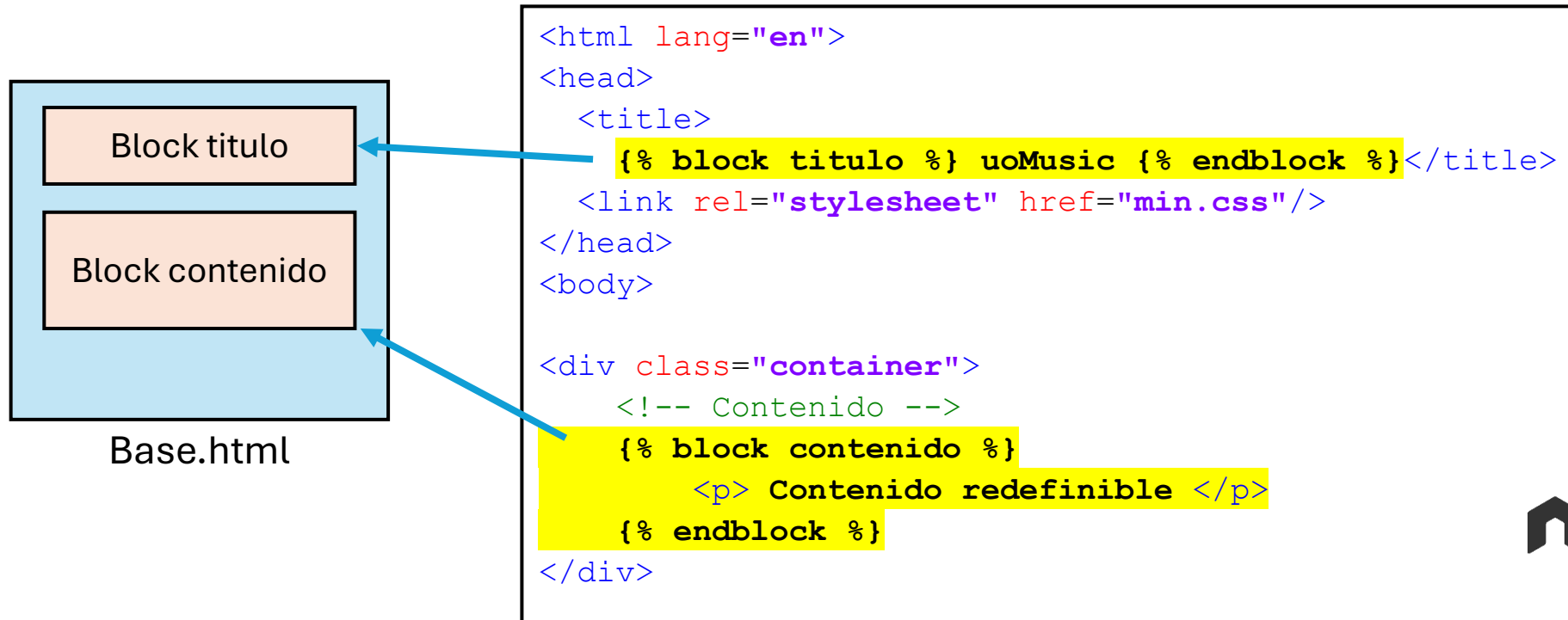
Base.html



Express > Vistas y plantillas

- **Composición de plantillas**

- Ejemplo, declaración de bloques en plantilla **base.html**



Express > Vistas y plantillas

- **Definición de plantillas**

- El resto de plantillas pueden extender de una plantilla base

```
{% extends "<path_plantilla>" %}
```

- Pueden redefinir o no el contenido de los bloques definidos en la plantilla base

```
{% block nombre %}  
    nuevo contenido  
{% endblock %}
```

```
{% extends "base.html" %}  
  
{% block titulo %}Tienda{% endblock %}  
  
{% block contenido %}  
<h2>Canciones</h2>  
<ul>  
    {% for song in songs %}  
        <li>{{ song.name }} </li>  
    {% endfor %}  
</ul>  
{% endblock %}
```



Express > Vistas y plantillas

- **Renderización de plantillas en la aplicación**

- Para utilizar el motor de plantillas **Twig** en una aplicación Express es necesario **incluirlo en la aplicación**.
- El módulo de Twig se incorpora en el archivo: app.js

```
let express = require('express');
let app = express();
...

// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'twig');
```



Express > Vistas y plantillas

- **Renderización de plantillas en la aplicación**

- El objeto **twig** proporciona una función:
 - **render(<plantilla>, <modelo de datos>)** que genera el **HTML resultante** a partir de la plantilla.
- El **modelo de datos** es un objeto formado por pares **clave : valor**
 - Los valores pueden ser tipos simples, objetos, o colecciones
- El código HTML generado por **render()** normalmente se envía como respuesta al cliente

```
app.get("/songs", function (req, res) {  
  songs = [  
    {'title': 'Blank space', 'price': '1.2'},  
    {'title': 'See you again', 'price': '1.3'},  
    {'title': 'Uptown Funk', 'price': '1.1'}  
  ]  
  let response = {  
    seller: 'Tienda de canciones',  
    songs: songs  
  };  
  res.render('test', response);  
});
```

```
<h1> {{ seller }} </h1>  
  
{% for song in songs %}  
  <li>{{ song.name }} </li>  
{% endfor %}
```

test.twig





Escuela de Ingeniería Informática

Escuela de Ingeniería Informática
School of Computer Science Engineering

Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Sistemas Distribuidos e Internet

Introducción a Node.js

Parte 1

Desarrollo de aplicaciones y servicios web modernos
con JavaScript



Dr. Edward Rolando Núñez Valdez

nunezedward@uniovi.es